ONNX·TensorRT Day 1

ONNX 첫걸음 — 내보내고, 보고, 실행하기

HYUNDAI AI Insight Campus · 온디바이스 AI · ONNX·TensorRT 1/5

2026. 11. 4 (수) 09:00–18:00 · 8교시 · Google Colab (하드웨어 없음)

export | Netron | ONNX Runtime | allclose | NCHW | 전처리

강사 : 김정인 · 김구수

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1 왜 ONNX 인가

학습하는 곳과 실행하는 곳이 다른 이유 , .pt 와 .onnx 의 차이를 말한다

2 첫 ONNX 만들기

nn.Linear(3, 2) 를 export 하고 파일 크기로 내용을 짐작한다 (8 × 4 B = 32 B)

3 Netron 으로 보기

노드 · 화살표 · 가중치 · 입출력 shape 를 읽는다 — Linear → Gemm

4 ONNX Runtime 실행

InferenceSession → run → np.allclose 로 PyTorch 와 같은지 확인한다

5 이름 · shape

input / logits 이름을 붙이고 (N, C, H, W) 를 맞춘다

6 사진 한 장 추론

전처리 6 단계로 MobileNetV3 top-5 를 구하고 , 처음 보는 모델에 4 단계를 혼자 적용한다

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 매 교시 보기 → 따라하기 → 바꿔보기

교시	시간	주제	형태	애니메이션
1	09:00–09:50	왜 ONNX 인가 — 학습하는 곳 vs 실행하는 곳	강의 + 실습	D1-1 학습 vs 실행
2	10:00–10:50	첫 ONNX 만들기 — nn.Linear(3, 2) 내보내기	강의 + 실습	D1-2 export 는 녹화다
3	11:00–11:50	Netron 으로 보기 — 노드 · 화살표 · 가중치	강의 + 실습	D1-3 Netron 그래프 읽기
4	13:00–13:50	ONNX Runtime 으로 실행 — PyTorch 와 같은가 ?	강의 + 실습	D1-4 InferenceSession
5	14:00–14:50	이름 붙이기 · 입력 shape (N, C, H, W)	강의 + 실습	D1-5 입력 shape 맞추기
6	15:00–15:50	MNIST CNN 내보내기 — 같은 4 단계를 큰 모델에	강의 + 실습	D1-6 CNN 그래프와 shape
7	16:00–16:50	이미지 한 장 추론 — 전처리 6 단계 · MobileNetV3	강의 + 실습	D1-7 사진 → 입력 텐서
8	17:00–17:50	미니 프로젝트 — 혼자서 4 단계 · 채점	개인 실습	D1-8 4 단계 정리

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 확인 (노트북) → 점검 (워크시트) — 모두 브라우저에서 바로 열립니다

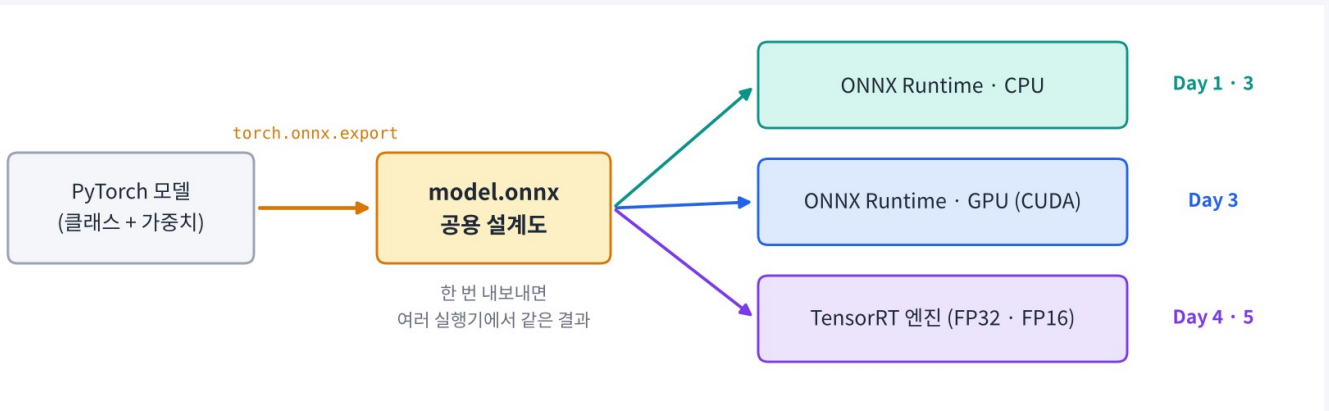
기본 주소 (Day 1)

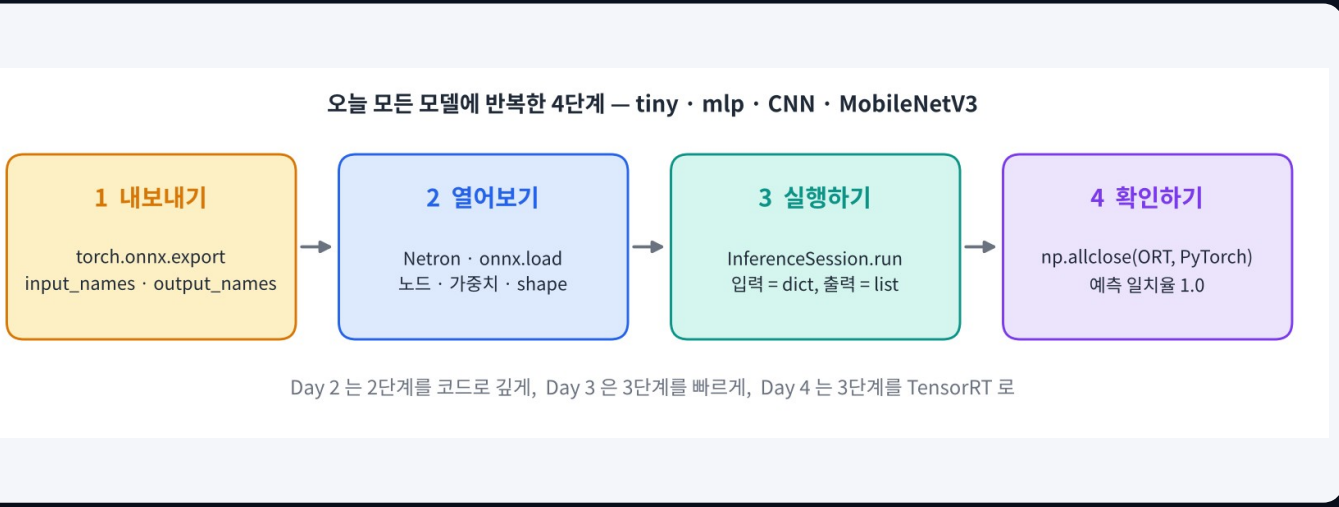
https://imguru-mooc.github.io/AI_CAMPUS/OT-day1/

animation.html	애니메이션 8 개 (D1-1 ~ D1-8) — #d1-1 처럼 바로가기
worksheet.html	개념 워크시트 — #t1 ~ #t8 교시 바로가기 (교시당 10 문항)
OT_Day1_lecture.ipynb	강의 노트북 — 1~8 교시 문제 + 미니 프로젝트 (Colab 에서 열기)
OT_Day1.pdf	오늘 슬라이드 PDF
data/mnist_cnn.pt · mnist_sample.npz	6 교시 — 학습된 SimpleConvNet 가중치 · MNIST 100 장
data/butterfly · orange · baboon.jpg	7 · 8 교시 — 사진 3 장 (OpenCV 예제)
https://netron.app	Netron — .onnx 파일을 끌어다 놓으면 그래프가 보인다

오늘의 큰 그림 — 같은 4 단계를 다섯 모델에

내보내기 → 열어보기 → 실행하기 → 확인하기 · tiny → mlp → CNN → MobileNetV3 → SafetyNet





오늘 만나는 모델

tiny — 2–5 교시
nn.Linear(3, 2) · 8 파라미터

mlp — 3 교시
Linear → ReLU → Linear · 26

SimpleConvNet — 6 교시
MNIST · 433,890 · 1.74 MB

MobileNetV3-S — 7 교시
ImageNet 1000 · 224×224

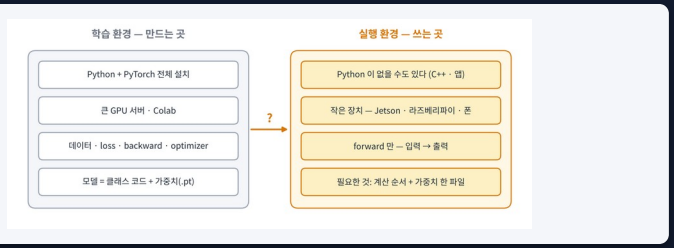
SafetyNet — 8 교시
64×64 · 3 클래스 · 1,491

1 왜 ONNX 인가 — 학습 vs 실행

09:00–09:50

학습 목표

- 학습 환경 (Colab · PyTorch) ≠ 실행 환경 (작은 장치)
- 실행에 필요한 것 = 계산 순서 + 가중치
- .pt 는 가중치만 , ONNX 는 둘 다 한 파일
- ONNX 에 담기는 것은 forward 뿐



노트북 실습

1. 문제 1-1 tiny.pt 열어 key · shape 보기
2. 문제 1-2 추론에 필요한 단계 고르기

▶ 애니메이션 D1-1 학습 vs 실행

핵심 개념

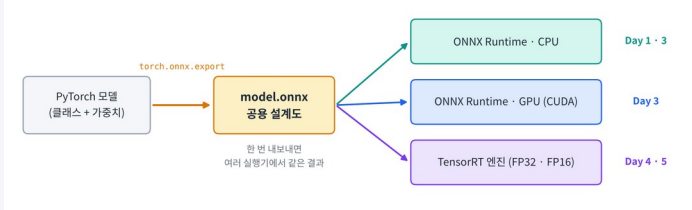
state_dict

```
sd = torch.load("tiny.pt")
list(sd.keys())  # ['weight', 'bias']
```

추론 단계

```
전처리 → model(x) → argmax
(loss · backward · step 은 학습만 )
```

길



체크포인트

tiny.pt 에는 weight (2, 3) · bias (2) 숫자 8 개만 있다 . 이 숫자로 무엇을 계산하는지는 파이썬 코드에 있다 — ONNX 는 그래프까지 저장한다 .

2 첫 ONNX 만들기 — nn.Linear(3, 2)

10:00–10:50

학습 목표

- ① 모델 ② 더미 입력 ③ export ④ 파일 확인
- export = 더미 입력으로 한 번 실행하며 녹화
- 더미 입력은 값이 아니라 shape · dtype
- 파일 ≈ 파라미터 × 4 B + 그래프 설명



노트북 실습

1. 문제 2-1 더미 입력 (1, 3)
2. 문제 2-2 torch.onnx.export
3. 문제 2-3 파라미터 8 개 × 4 B
4. 🔁 출력 2 → 5 바꿔보기

▶ 애니메이션 D1-2 export 는 녹화다

핵심 개념

손계산

```
W = [[1,2,3],[0,-1,1]], b = [0.5,-0.5]
tiny([1,1,1]) = [6.5, -0.5]
```

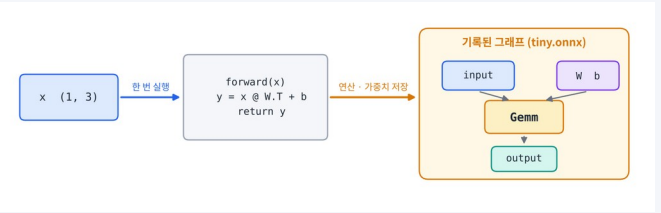
export

```
dummy = torch.randn(1, 3)
torch.onnx.export(tiny, dummy, "tiny.onnx", **EXPORT_KW)
```

파일 크기

```
8 × 4 B = 32 B  |  tiny.onnx 249 B
Linear(3, 5): 20 × 4 = 80 B | 297 B
```

녹화



체크포인트

EXPORT_KW = 기존 (TorchScript) 방식 export 옵션. 오늘은 모든 export 에 그대로 붙인다.

3 Netron 으로 보기 — 노드 · 화살표 · 가중치

11:00–11:50

학습 목표

- 상자 = 노드 (연산), 화살표 = 데이터 흐름
- B · C = 가중치 (initializer)
- nn.Linear → Gemm (transB = 1)
- 코드로도 : graph.node · graph.initializer



노트북 실습

1. 문제 3-1 노드 종류 op_type
2. 문제 3-2 weight (2, 3) = B ⟨2×3⟩
3. 문제 3-3 mlp 노드 예측

▶ 애니메이션 D1-3 Netron 그래프 읽기

핵심 개념

Colab 에서

```
show_netron("tiny.onnx", port=8081)
# 안 보이면 netron.app 에 끌어다 놓기
```

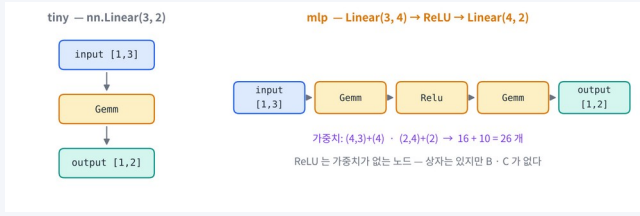
코드로

```
m = onnx.load("tiny.onnx")
[n.op_type for n in m.graph.node]  # ['Gemm']
```

mlp

```
Linear(3,4) → ReLU → Linear(4,2)
['Gemm', 'Relu', 'Gemm'] · 가중치 26 개
```

층 쌓기



체크포인트

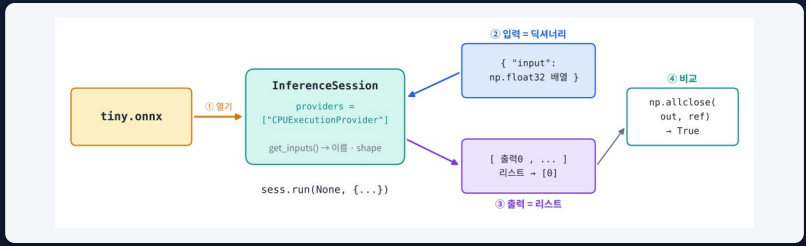
자동 입력 이름은 onnx::Gemm_0, 출력은 3 — 읽기 어렵다 . 5 교시에 input / output 으로 바꾼다 .

4 ONNX Runtime 으로 실행 — PyTorch 와 같은가 ?

13:00–13:50

학습 목표

- InferenceSession(path, providers)
  - 입력 = { 이름 : np.float32 배열 }
  - 출력 = 리스트 → [0]
- == 대신 np.allclose — 끝자리 차이는 정상



노트북 실습

1. 문제 4-1 CPUExecutionProvider
2. 문제 4-2 get_inputs()[0].name
3. 문제 4-3 run → [6.5, -0.5]
4. 문제 4-4 20 회 allclose

▶ 애니메이션 D1-4 InferenceSession 한 번 실행

핵심 개념

실행

```
sess = ort.InferenceSession("tiny.onnx",
    providers=["CPUExecutionProvider"])
y = sess.run(None, {in_name: x.numpy()})[0]
```

비교

```
np.allclose(out, ref, atol=1e-5)  # True
```

float64

```
INVALID_ARGUMENT : Unexpected input data type.
Actual: (tensor(double)), expected: (tensor(float))
```

이름 틀림

```
Required inputs (['onnx::Gemm_0']) are missing
from input feed (['x'])
```

체크포인트

내보낸 뒤 반드시 PyTorch 와 비교한다 — 오늘 가장 중요한 습관 .

5 이름 붙이기 · 입력 shape (N, C, H, W)

14:00–14:50

학습 목표

- input_names · output_names (리스트)
  - 이미지 텐서 = (N, C, H, W), 1 장도 N = 1
- OpenCV 는 (H, W, C) — 순서가 다르다
  - 입력 shape 는 export 때 고정된다



노트북 실습

1. 문제 5-1 input / output 이름
2. 문제 5-2 (1,1,28,28) · (1,3,224,224) · (8,3,224,224)
3. 문제 5-3 배치 4 → 오류 읽기

▶ 애니메이션 D1-5 입력 shape 맞추기

핵심 개념

이름

```
torch.onnx.export(tiny, dummy, "tiny_named.onnx",
  input_names=["input"], output_names=["output"])
```

배치 4

```
Got: 4 Expected: 1  (index 0 = 배치 축 )
→ dynamic_axes 는 Day 2
```

이름 비교



체크포인트

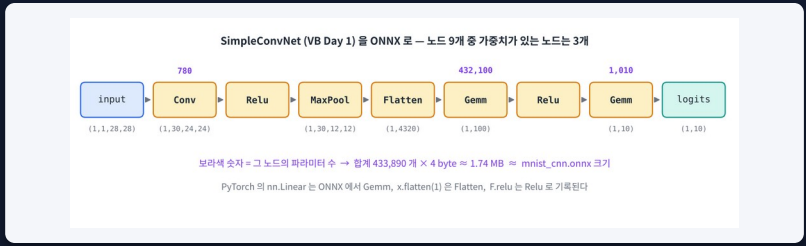
이 과정의 분류 모델 이름 규칙 : 입력 input, 출력 logits (VB Day 5 와 같음).

6 MNIST CNN 내보내기 — 같은 4 단계를 큰 모델에

15:00–15:50

학습 목표

- VB Day 1 SimpleConvNet · 학습된 가중치 제공
- Conv · Relu · MaxPool · Flatten · Gemm · Relu · Gemm
  - 가중치 노드 3 개 · 433,890 × 4 B ≈ 1.74 MB
  - 전처리가 다르면 오류 없이 틀린다



노트북 실습

1. 문제 6-1 export (1,1,28,28) · input/logits
2. 문제 6-2 노드 · 가중치 노드 3 개
3. 문제 6-3 100 장 일치율 1.0
4. 🔁 /255 빼먹기

▶ 애니메이션 D1-6 CNN 그래프와 shape

핵심 개념

export

```
torch.onnx.export(cnn, torch.zeros(1, 1, 28, 28),
"mnist_cnn.onnx",
  input_names=["input"], output_names=["logits"])
```

한 장씩

```
x = (img.astype(np.float32) / 255)[None, None]
sess_cnn.run(None, {"input": x})[0].argmax()
```

결과 (실행 예)

```
ORT–PyTorch 일치율 1.000 · 정확도 1.000
/255 없이 정확도 0.92 – 오류 메시지 없음
```

체크포인트

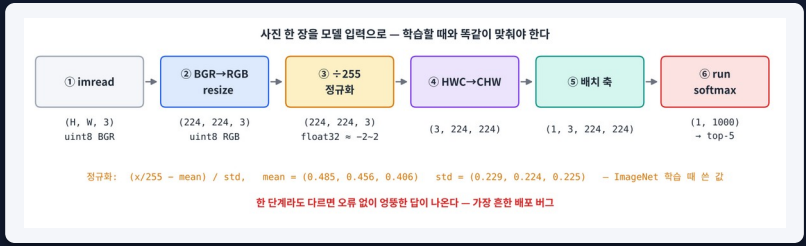
분류 모델의 두 번째 확인 기준 = 예측 (argmax) 일치율 . logits 끝자리가 달라도 답은 같아야 한다 .

7 이미지 한 장 추론 — 전처리 6 단계 · MobileNetV3

16:00–16:50

학습 목표

- mnv3s.onnx — ImageNet 1000 클래스 , 입력 224
- imread → RGB · resize → 정규화 → CHW → 배치
- softmax → top-5 · 클래스 이름
- preprocess · classify 함수로 묶기



노트북 실습

1. 문제 7-1 BGR→RGB · resize (W, H)
2. 문제 7-2 /255 · mean · std
3. 문제 7-3 transpose(2, 0, 1) · [None]
4. 문제 7-4 softmax · top-5

▶ 애니메이션 D1-7 사진 → 입력 텐서

핵심 개념

①②

```
img = cv2.imread("butterfly.jpg")      # (356, 493, 3) BGR
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = cv2.resize(rgb, (224, 224))        # (W, H)
```

③

```
x = (img.astype(np.float32) / 255 - MEAN) / STD
MEAN (0.485, 0.456, 0.406) · STD (0.229, 0.224, 0.225)
```

④⑤⑥

```
x_in = x.transpose(2, 0, 1)[None]        # (1, 3, 224, 224)
logits = sess_mn.run(None, {"input": x_in})[0][0]
```

체크포인트

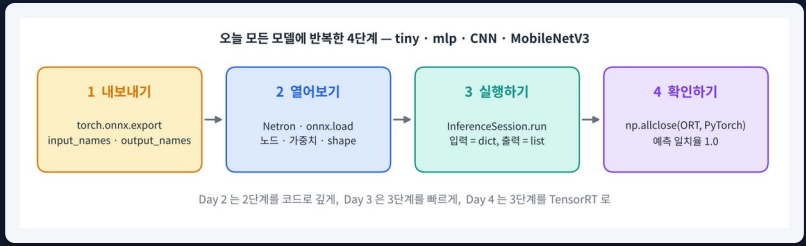
reshape 는 모양만 , transpose 는 축 순서를 바꾼다 . 전처리는 ONNX 밖의 코드 — 학습 때와 똑같이 .

8 미니 프로젝트 — 혼자서 4 단계 · 채점

17:00–17:50

학습 목표

- 처음 보는 SafetyNet (64×64 → 3 클래스)
- eval → export → 요약 → 실행 · 비교
- onnx_report(path) 함수 만들기
- eval export 는 Conv 뒤 BN 을 합친다



노트북 실습

1. 과제 1 export (20)
2. 과제 2 그래프 요약 (20)
3. 과제 3 ORT · 비교 (20)
4. 과제 4 onnx_report (20)
5. 과제 5 · 6 사진 3 장 · 해석 (20)

▶ 애니메이션 D1-8 4 단계 정리

핵심 개념

SafetyNet

```
Conv-BN-ReLU-MaxPool × 2 → AvgPool → Linear(16, 3)
파라미터 1,491 · ONNX 노드 9 개 · BN 노드 없음
```

onnx_report

```
{'size_kb', 'n_nodes', 'ops', 'inputs', 'outputs'}
inputs = [('input', [1, 3, 64, 64])]
```

배포 = 3 가지

```
ONNX 파일 + 전처리 규칙 + 클래스 이름
```

체크포인트

채점 셀 자동 90 점 + 과제 6 서술 10 점 . 채점 셀 결과가 보이는 상태로 노트북을 공유해 제출 .

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

왜 ONNX	.pt = 가중치만 · ONNX = 계산 순서 + 가중치 한 파일 · forward 만 담긴다	D1-1
export	더미 입력으로 한 번 실행하며 녹화 — 값이 아니라 shape · dtype · 8 × 4 B = 32 B	D1-2
Netron	상자 = 노드, 화살표 = 흐름, B · C = 가중치 · Linear → Gemm (transB = 1)	D1-3
ONNX Runtime	InferenceSession → run(None, {name: float32})[0] → np.allclose	D1-4
이름 · shape	input / logits · (N, C, H, W) · 입력 shape 는 export 때 고정	D1-5
CNN	Conv · Relu · MaxPool · Flatten · Gemm × 2 · 433,890 ≈ 1.74 MB · 일치율 1.0	D1-6
전처리	imread → RGB · resize(W, H) → /255 · 정규화 → transpose(2, 0, 1) → [None]	D1-7
4 단계	eval → export → 열어보기 · onnx_report → 실행 · 확인 · BN 은 Conv 에 합쳐진다	D1-8

Day 2 로 가는 다리 — ONNX 속 들여다보기

오늘 Netron 으로 본 것을 코드로, 오늘 막힌 '배치 고정' 을 푼다

Day 1 (오늘)		Day 2 (11/5 목)
Netron 으로 그래프 보기	→	describe_onnx() — 노드 · 가중치 · 크기를 표로
graph.node · initializer 맛보기	→	opset · checker · shape_inference
배치 4 → Got: 4 Expected: 1	→	dynamic_axes 로 배치 1 · 4 · 8 실행
오류 없는 export 만	→	일부러 실패하는 export 3 가지 고치기
SafetyNet 노드 9 개	→	onnxsim 으로 그래프 단순화 · 노드 수 비교
preprocess · onnx_report	→	그대로 사용 — Drive 에 사본 저장

내보내기 → 열어보기 → 실행하기 → 확인하기 — 내일은 '열어보기' 를 코드로 깊게